

## Chapter 5: Flow Control

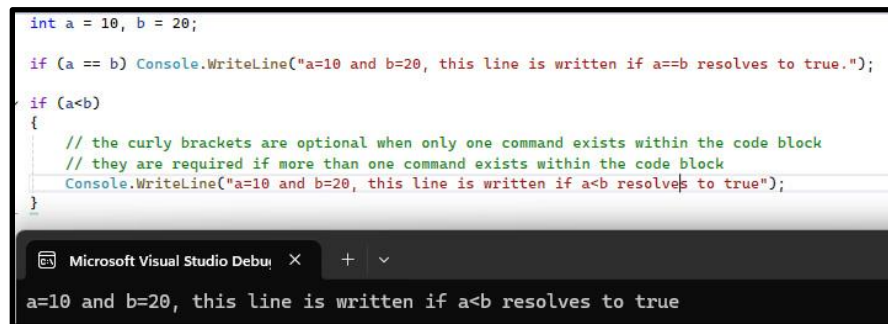
Flow control refers to the order in which commands and/or subprograms are executed within an application. The typical situation is something like “execute this section of code if some conditional is true” or “execute this subprogram a specified number of times” or “execute this section of code once for each member of a set”. Super important stuff because elegant flow control is a cornerstone to good programming.

Okay, let’s jump to the inside and take a look at some flow control examples.

Please begin by creating a new console application project named chapter 5 flow control.

We will start with a simple if statement. (Figure 5.1)

**Figure 5.1: if**



```
int a = 10, b = 20;

if (a == b) Console.WriteLine("a=10 and b=20, this line is written if a==b resolves to true.");

if (a<b)
{
    // the curly brackets are optional when only one command exists within the code block
    // they are required if more than one command exists within the code block
    Console.WriteLine("a=10 and b=20, this line is written if a<b resolves to true");
}
```

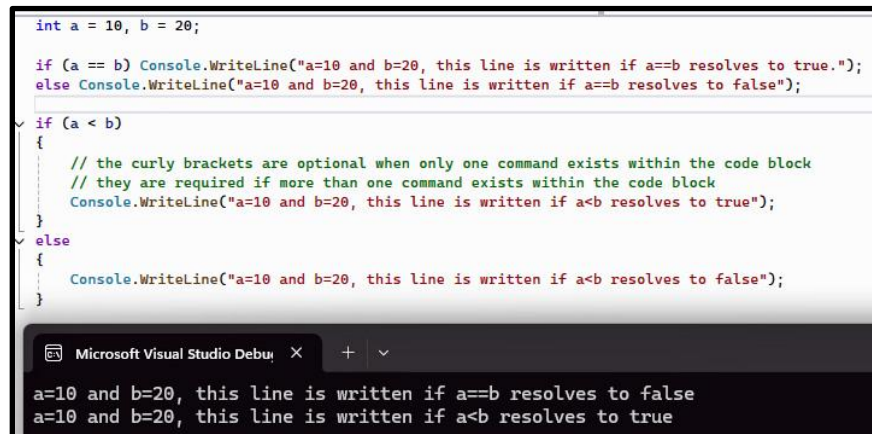
Microsoft Visual Studio Debug Console: a=10 and b=20, this line is written if a<b resolves to true

So, what did we learn?

- First, that the equality test in C# is represented by a pair of equal signs.
- Next, that an if command tests a conditional and when that conditional resolves to true, the subsequent command or commands are executed. If the conditional resolves to false, the subsequent command or commands are not executed. In our example here the first Console.WriteLine command is not executed because variable a does not equal variable b. The second Console.WriteLine command is executed because variable a is less than variable b.
- Finally, we learned that the curly brackets are optional if the executed code block includes only a single command but are required if the executed code block contains more than one command. Net, the curly brackets delimit the to-be-executed code.

Next let's update our code to include the else keyword. (Figure 5.2)

**Figure 5.2: if else Syntax**



```
int a = 10, b = 20;

if (a == b) Console.WriteLine("a=10 and b=20, this line is written if a==b resolves to true.");
else Console.WriteLine("a=10 and b=20, this line is written if a==b resolves to false");

if (a < b)
{
    // the curly brackets are optional when only one command exists within the code block
    // they are required if more than one command exists within the code block
    Console.WriteLine("a=10 and b=20, this line is written if a<b resolves to true");
}
else
{
    Console.WriteLine("a=10 and b=20, this line is written if a<b resolves to false");
}
```

Microsoft Visual Studio Debug Console Output:

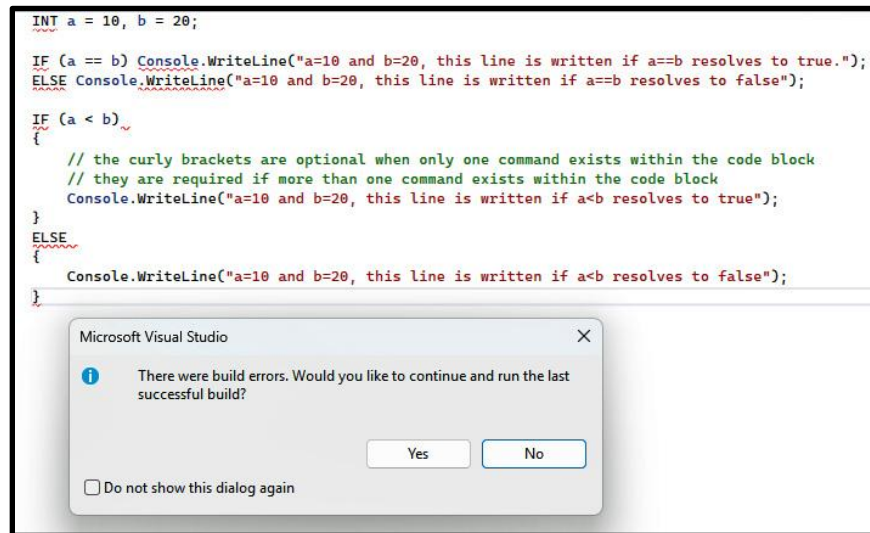
```
a=10 and b=20, this line is written if a==b resolves to false
a=10 and b=20, this line is written if a<b resolves to true
```

So, what did we learn from this example?

- That the else code block is executed when the original if conditional resolves to false.

Now let's change our code to have all the keywords in uppercase. (Figure 5.3)

**Figure 5.3: Keywords Are All Lowercase**

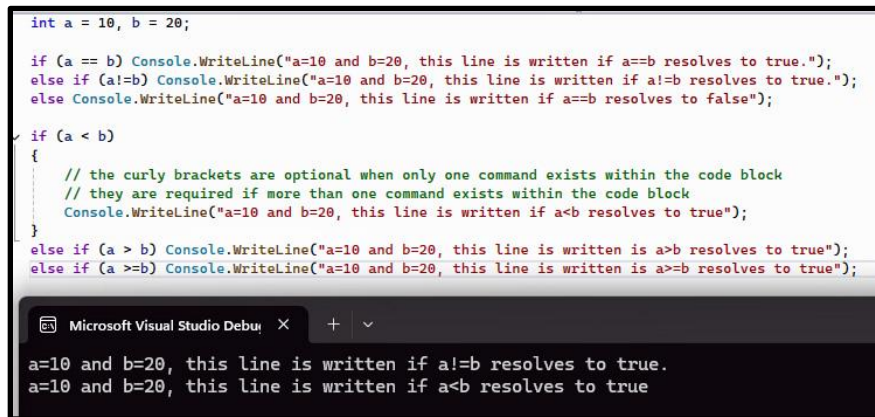


What did we learn from this example?

- That the C# compiler did not recognize the `INT`, `IF` and `ELSE` keywords when they were in uppercase. C# has 79 keywords, each is lowercase.
- We also learned that intellisense recognized the problem and underlined the improperly formatted keywords in red.
- Finally, please note the formatting of the `Console.WriteLine` method. The use of a capital letter at the beginning of each word is known as PascalCase. Microsoft uses this format in all its preprogrammed methods. By the way, this dot notation command says that we need to go to the `Console` class and once there execute the `WriteLine` method.

In our next example we include the `else if` syntax. (Figure 5.4) In the `if`, `else if`, `else` syntax the `if` conditional is evaluated first. If it resolves to `true`, the code block associated with it is executed and the remaining `else if` and `else` conditionals are ignored. When the `if` conditional resolves to `false` the flow control moves to the first `else if` conditional. If that conditional resolves to `true`, then the code block associated with it is executed and the remaining code is ignored. Any number of `else if` conditionals may exist and they are evaluated top to bottom. Only the code block associated with the first one which resolves to `true` is executed. If none of the `if` or `else if` conditionals resolves to `true`, then the `else` code block is executed. The `else` block is optional. I have included it here in our first command block but excluded it in our second.

Figure 5.4: else if



```
int a = 10, b = 20;

if (a == b) Console.WriteLine("a=10 and b=20, this line is written if a==b resolves to true.");
else if (a!=b) Console.WriteLine("a=10 and b=20, this line is written if a!=b resolves to true.");
else Console.WriteLine("a=10 and b=20, this line is written if a==b resolves to false");

if (a < b)
{
    // the curly brackets are optional when only one command exists within the code block
    // they are required if more than one command exists within the code block
    Console.WriteLine("a=10 and b=20, this line is written if a<b resolves to true");
}

else if (a > b) Console.WriteLine("a=10 and b=20, this line is written is a>b resolves to true");
else if (a >=b) Console.WriteLine("a=10 and b=20, this line is written is a>=b resolves to true");
```

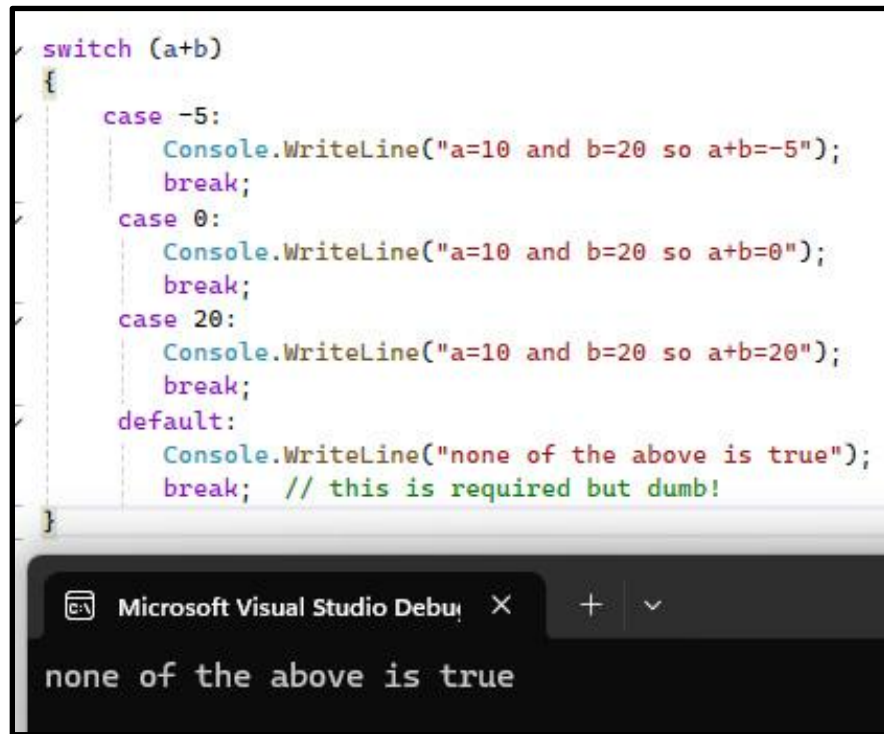
Microsoft Visual Studio Debug Console

```
a=10 and b=20, this line is written if a!=b resolves to true.
a=10 and b=20, this line is written if a<b resolves to true
```

We move next to the switch/case flow control syntax. (Figure 5.5) I think the code here is pretty self-explanatory other than the break keyword. The break keyword simply says stop what you are doing within this block of code and jump to the bottom of the parent command. In our example, that parent command is the switch keyword. By the way, I coded for some 20+ years in C and C# and never used or even saw the switch command. Everybody uses the if syntax instead. However, I know some of you will be taking certification exams as associated with this content, so I include the switch/case command just in case this syntax lives on one of those tests. As an aside, I believe my comment regarding the required break command with the default keyword is accurate. Think it through, the only time that should be required is when the default keyword precedes one of the case options and that would be just too weird. Net, stick with the if syntax.

By the way, I have commented out all my if syntax at this point to keep the output clean. I did however leave in my a and b variable declarations and assignments.

Figure 5.5: switch/case



```
switch (a+b)
{
    case -5:
        Console.WriteLine("a=10 and b=20 so a+b=-5");
        break;
    case 0:
        Console.WriteLine("a=10 and b=20 so a+b=0");
        break;
    case 20:
        Console.WriteLine("a=10 and b=20 so a+b=20");
        break;
    default:
        Console.WriteLine("none of the above is true");
        break; // this is required but dumb!
}
```

Microsoft Visual Studio Debug Console

none of the above is true

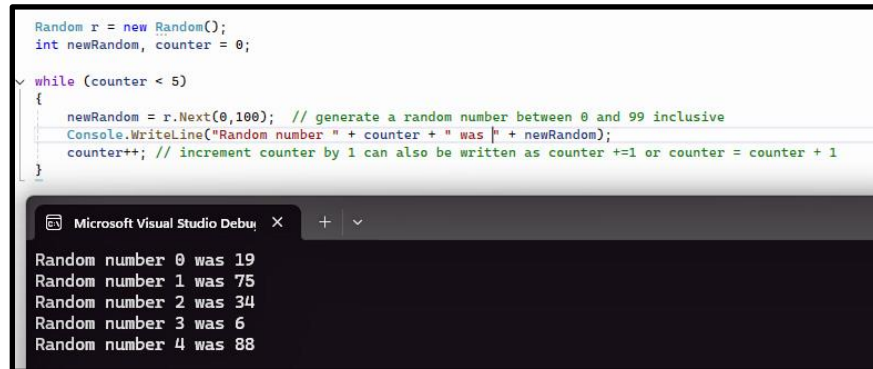
In our next example we introduce the while keyword. (Figure 5.6) The while syntax executes a block of code while a conditional resolves to true. In our example the block of code is executed while a variable named counter is less than 5.

What did we learn from this example?

- We learned how to use the while keyword to control the number of times a block of code is executed.
- We also learned about the preprogrammed class named Random. We will learn more about classes in chapters 7 and 8. For now just know that a class is built up from data type primitives or other classes and may have properties (the typical person class for example will have properties of first and last name, gender, email address, etc.) and methods (things they can do or execute, in the case of our Random class it has a method which can return a random number between two specified input parameters).
- We also learned two new ways to increment a variable by one. Of course we could have written "counter = counter + 1;" but we chose instead to write the most abbreviated version of that syntax as "counter++;". Please also note my comment which says there is even a third form of this syntax which reads "counter +=1;"

What do you suppose happens if you forget to increment the counter variable in this example? Yes of course, the program continues to execute indefinitely. This scenario is called an infinite loop, and they are most definitely not cool. Easy enough to just shut down Visual Studio if you are working on your own box but if you are working on a big server farm or connected to a database it can be very embarrassing. Some systems administrator needs to physically shut down your process, not cool.

**Figure 5.6: while**



```
Random r = new Random();
int newRandom, counter = 0;

while (counter < 5)
{
    newRandom = r.Next(0,100); // generate a random number between 0 and 99 inclusive
    Console.WriteLine("Random number " + counter + " was " + newRandom);
    counter++; // increment counter by 1 can also be written as counter +=1 or counter = counter + 1
}
```

Microsoft Visual Studio Debug Console

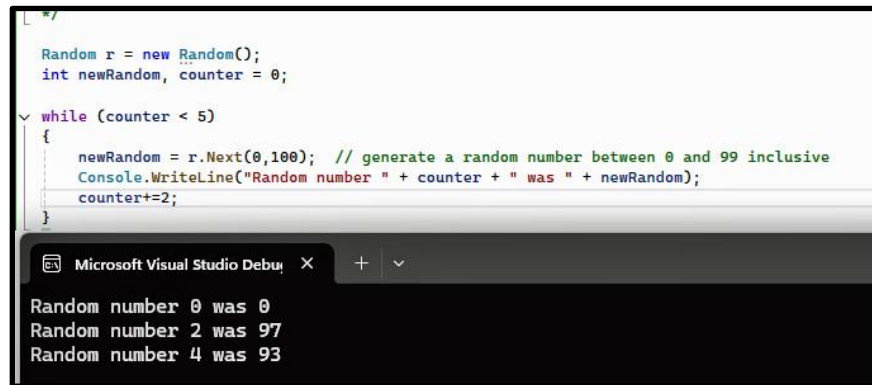
```
Random number 0 was 19
Random number 1 was 75
Random number 2 was 34
Random number 3 was 6
Random number 4 was 88
```

Okay, do me a favor and tell yourself without looking at the output what this next example should return. (Figure 5.7) If you said random numbers associated with the values 0, 2 and 4 then you are starting to think like a programmer...well done.

What did we learn from this example?

- That the variable `+= X` syntax says take variable and increment it by X with each pass of the loop. Okay, cool.

Figure 5.7: += syntax



```
Random r = new Random();
int newRandom, counter = 0;

while (counter < 5)
{
    newRandom = r.Next(0,100); // generate a random number between 0 and 99 inclusive
    Console.WriteLine("Random number " + counter + " was " + newRandom);
    counter+=2;
}
```

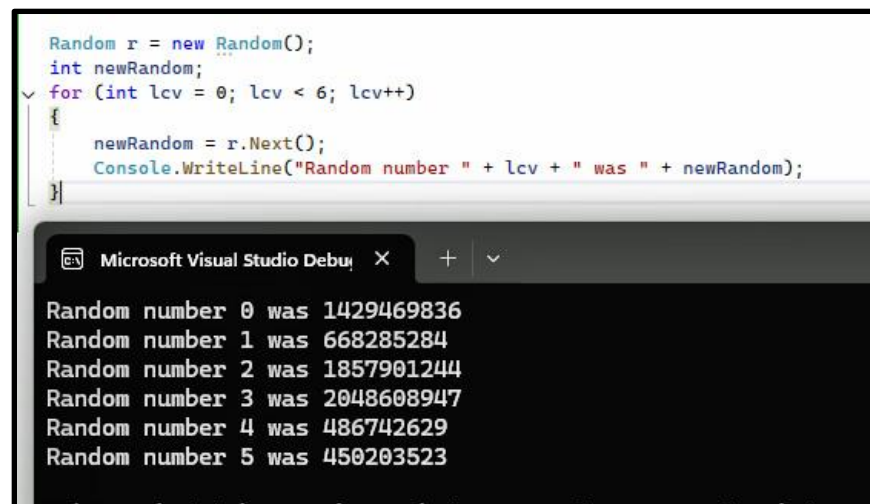
Microsoft Visual Studio Debug Console

```
Random number 0 was 0
Random number 2 was 97
Random number 4 was 93
```

In our next example we exchange the while keyword with the for keyword. (Figure 5.8) This guy requires some explanation.

- First, the for loop as shown here takes three parameters. The first in our example is an integer variable named lcv which in our case is initialized to zero. By the way, my choice here of a variable name was not an accident as this variable is generally known as the “loop control variable”. The second parameter is a conditional. The loop is executed while that conditional resolves to true. The third, and final parameter, is used to change the loop control variable. In our case here, increment it by one with each execution of the code block.
- Next, check out what happens when you feed the Random class Next method zero parameters. Sure, it feels free to return any integer. By the way, most random number generators like this do modular arithmetic on the number of seconds past midnight. Just a fun fact to know and share at your next “geek party.”

Figure 5.8: for loop



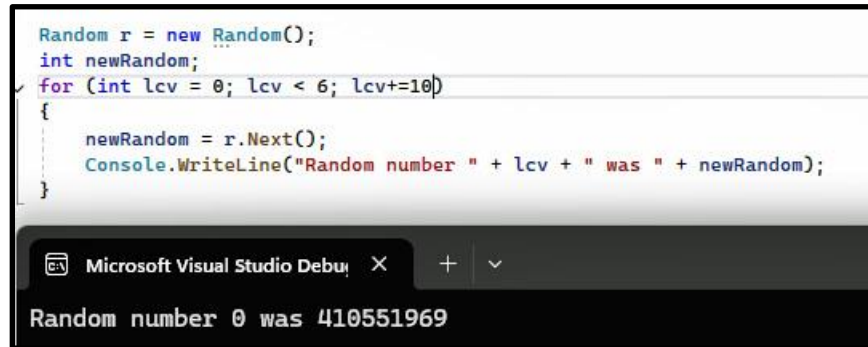
```
Random r = new Random();
int newRandom;
for (int lcv = 0; lcv < 6; lcv++)
{
    newRandom = r.Next();
    Console.WriteLine("Random number " + lcv + " was " + newRandom);
}
```

Microsoft Visual Studio Debug Console

```
Random number 0 was 1429469836
Random number 1 was 668285284
Random number 2 was 1857901244
Random number 3 was 2048608947
Random number 4 was 486742629
Random number 5 was 450203523
```

In our next example (Figure 5.9) I used a random number seed, passed one parameter to the Next method of the Random class and changed the increment on my loop control variable to 10.

**Figure 5.9: Updated for loop**



```
Random r = new Random();
int newRandom;
for (int lcv = 0; lcv < 6; lcv+=10)
{
    newRandom = r.Next();
    Console.WriteLine("Random number " + lcv + " was " + newRandom);
}
```

Microsoft Visual Studio Debug Console

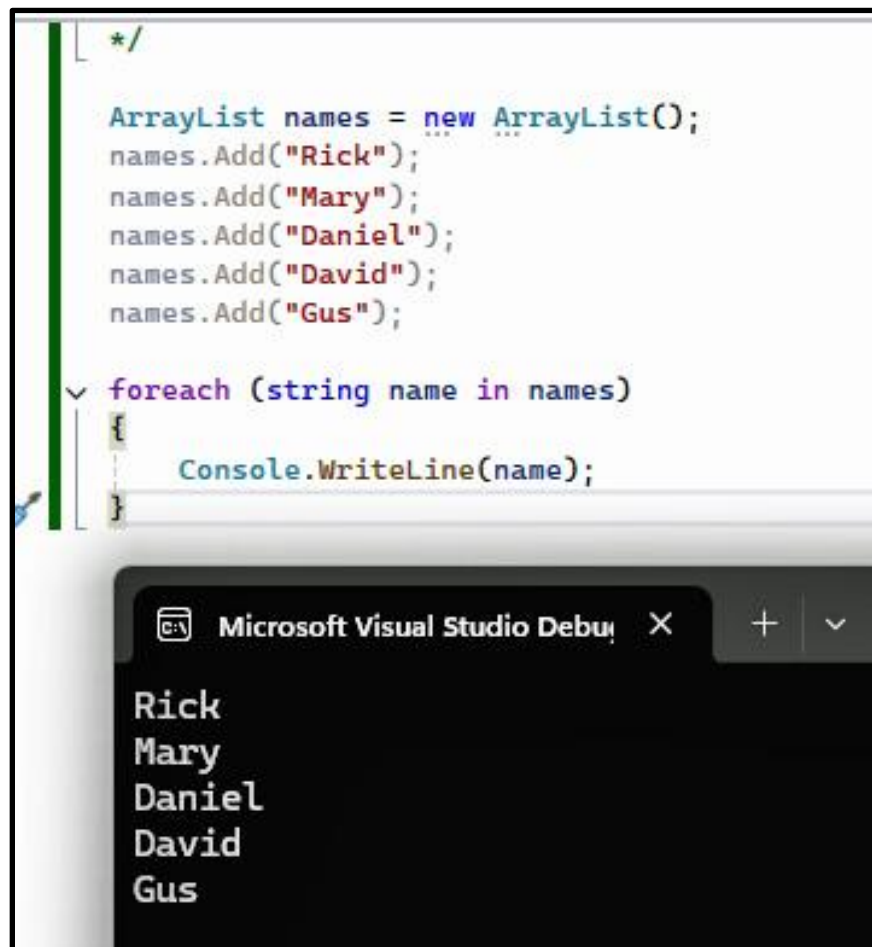
Random number 0 was 410551969

What did we learn?

- That we can seed the Random class Next method with a single parameter. If you repeatedly use the same seed, you will always get the same sequence of random numbers. Not important in this example but certainly very useful in other applications.
- That we can increment our loop control variable by any amount as long as the increment (or even decrement) is of the same data type as the loop control variable. Given this, do you suppose there are situations where your loop control variable is a double and the increment value is also a real number? Sure.

In our final flow control example we utilize an ArrayList class called names. (Figure 5.10) We then use the Add method of the ArrayList class to create five instances of the names class. Check out the foreach syntax. Do you suppose this is going to be valuable when we get to building classes and taking some actions on all the instances of that class? Sure.

Figure 5.10: foreach



```
*/  
  
ArrayList names = new ArrayList();  
names.Add("Rick");  
names.Add("Mary");  
names.Add("Daniel");  
names.Add("David");  
names.Add("Gus");  
  
foreach (string name in names)  
{  
    Console.WriteLine(name);  
}
```

Microsoft Visual Studio Debug Console

Rick  
Mary  
Daniel  
David  
Gus

Well, that was a pretty heavy-duty chapter. We learned all about the various methods for controlling the order commands and subroutines are executed within an application.

In particular, we learned the following syntaxes:

- If else if else
- switch / case
- while
- for
- foreach

This is the most important stuff we have covered so far. Please make certain you have a full understanding of how each of our examples worked and why we got the results we did. There is no recipe for how to build a large integrated software solution. You take these basic building blocks and use them to “imagine” that solution. You of course will start out just maintaining and modifying someone else’s code but after a few years you will be expected to create your

own. To do so these basic building blocks have to become second nature so you can focus on the true design of your solution.

In our next lesson we will examine some of the older methods for storing data within an application, i.e. collections. Once we have those under our belts we can move on to classes.